

Specyfikacja Techniczna Systemu

Investor Assistant

Dokumentacja Architektoniczna, Algorytmiczna i API Reference

Marek Kotulski

12 stycznia 2026

Spis treści

1	Architektura Systemu	3
2	Podstawy Matematyczne Modelu ANFIS	3
2.1	Warstwa 1: Fuzyfikacja (Fuzzification Layer)	3
2.2	Warstwa 2: Ewaluacja Reguł (Rule Antecedent Layer)	3
2.3	Warstwa 3: Normalizacja (Normalization Layer)	3
2.4	Warstwa 4: Wnioskowanie TSK (Consequent Layer)	3
2.5	Warstwa 5: Agregacja (Defuzzification Layer)	3
3	Specyfikacja Modułów Backend	4
3.1	Moduł: backend/data_engine.py	4
3.2	Moduł: backend/feature_factory.py	4
3.3	Moduł: backend/anfis_pytorch/anfis.py	4
3.4	Moduł: backend/neuro_engine.py	5
3.5	Moduł: backend/backtest_engine.py	5
3.6	Moduł: backend/optimizer_engine.py	5
4	Specyfikacja API REST (Flask)	6
4.1	Grupa: Konfiguracja i Dane	6
4.2	Grupa: Analiza i Trening	6
4.3	Grupa: Optymalizacja	6
5	Funkcje Pomocnicze (Utilities)	7
5.1	W pliku backend/app.py	7

1 Architektura Systemu

System zrealizowany jest w architekturze mikroserwisowej (monolitycznej logicznie), opartej na wzorcu MVC (Model-View-Controller).

- **Model:** Warstwa danych (**DataEngine**), Model ANFIS PyTorch (**AnfisNet**).
- **View:** Interfejs React.js (CSR - Client Side Rendering).
- **Controller:** Flask API (**app.py**) oraz orkiestratory logiki biznesowej (**NeuroEngine**, **BacktestEngine**).

2 Podstawy Matematyczne Modelu ANFIS

System wykorzystuje model **Adaptive Neuro-Fuzzy Inference System** typu Takagi-Sugeno-Kang (TSK) rzędu pierwszego. Poniżej przedstawiono formalny opis operacji tensorowych realizowanych przez sieć.

2.1 Warstwa 1: Fuzyfikacja (Fuzzification Layer)

Transformacja wektora wejściowego X na stopnie przynależności μ . Zastosowano różniczkowalną funkcję Gaussa.

$$O_{1,i} = \mu_{A_i}(x) = \exp\left(-\frac{(x - c_i)^2}{2\sigma_i^2}\right) \quad (1)$$

Gdzie: c_i (centroid) i σ_i (odchylenie) są parametrami trenowalnymi.

2.2 Warstwa 2: Ewaluacja Reguł (Rule Antecedent Layer)

Obliczenie siły aktywacji reguły (w_i) przy użyciu T-normy iloczynowej (Product T-norm).

$$O_{2,i} = w_i = \prod_{j=1}^n \mu_{A_j}(x_j) \quad (2)$$

2.3 Warstwa 3: Normalizacja (Normalization Layer)

Normalizacja sił reguł w celu zapewnienia stabilności numerycznej.

$$O_{3,i} = \bar{w}_i = \frac{w_i}{\sum_{k=1}^R w_k} \quad (3)$$

2.4 Warstwa 4: Wnioskowanie TSK (Consequent Layer)

Generacja wyjścia reguły jako liniowej kombinacji wejść (płaszczyzna).

$$O_{4,i} = f_i(x) = p_i x_1 + q_i x_2 + \dots + r_i \quad (4)$$

Parametry $\{p, q, r\}$ są optymalizowane metodą spadku gradientu.

2.5 Warstwa 5: Agregacja (Defuzzification Layer)

Obliczenie końcowej predykcji jako średniej ważonej.

$$Y_{pred} = \sum_i \bar{w}_i f_i(x) \quad (5)$$

3 Specyfikacja Modułów Backend

3.1 Moduł: backend/data_engine.py

Odpowiada za ETL (Extract, Transform, Load) oraz buforowanie danych.

- `DataEngine(class)`: Singleton zarządzający sesjami.
- `__init__(self, api_key)`: Inicjalizacja klucza FRED API.
- `_fetch_fred_direct(self, series_id) -> Series`: Pobiera surowe dane makroekonomiczne z API St. Louis Fed.
- `_get_market_data_cached(self, ticker, date_str) -> DataFrame`: Dekorator: `@lru_cache`. Pobiera OHLCV z Yahoo Finance, obsługuje `auto_adjust=False` i naprawia `MultiIndex`.
- `_get_macro_data_cached(self, date_str) -> DataFrame`: Dekorator: `@lru_cache`. Agreguje i przetwarza:
 - *T10Y2Y*: Spread obligacji (Krzywa dochodowości).
 - *VIXCLS*: Indeks zmienności.
 - *M2SL*: Podaż pieniądza (konwersja na zmianę r/r).
 - *CPIAUCSL*: Inflacja CPI (konwersja na zmianę r/r).
- `prepare_dataset(self, ticker) -> DataFrame`: Wykonuje *Left Join* danych rynkowych z makroekonomicznymi. Stosuje `ffill` dla danych miesięcznych (CPI, M2) mapowanych na dzienne notowania.

3.2 Moduł: backend/feature_factory.py

Statyczna fabryka inżynierii cech.

- `FeatureFactory(class)`
- `generate_features(df) -> DataFrame`: Oblicza wskaźniki techniczne:
 - *RSI (14)*: Wskaźnik siły względnej.
 - *MACD*: (12, 26, 9) - Moving Average Convergence Divergence.
 - *Price_ROC_{N}*: Rate of Change dla okien 10, 20, 50.
 - *Volat_{N}*: Rolling Standard Deviation dla okien 20, 50.
- `select_diverse_top_features(df, target_col, top_n, max_corr) -> (list, dict)`: Algorytm selekcji cech. Oblicza macierz korelacji cech z targetem (Pearson/Spearman). Następnie iteracyjnie wybiera najlepsze cechy, odrzucając te, które są współliniowe (skorelowane $> max_corr$) z już wybranymi.

3.3 Moduł: backend/anfis_pytorch/anfis.py

Implementacja sieci neuronowej w PyTorch.

- `FuzzifyVariable(nn.Module)`: Przechowuje parametry funkcji przynależności dla jednej zmiennej wejściowej.
- `FuzzifyLayer(nn.Module)`: Kontener dla wszystkich zmiennych wejściowych. Metoda `forward` zwraca tensor przynależności.

- `AntecedentLayer(nn.Module)`: Generuje kombinatorykę reguł (iloczyn kartezjański). Oblicza siłę odpalenia reguł za pomocą `torch.prod`.
- `ConsequentLayer(nn.Module)`: Realizuje warstwę TSK. Wykorzystuje **Tensor Broadcasting** do rozszerzenia wymiarów wejścia (B, R, I, 1) i wag (B, R, I, 0), co umożliwia równoległe obliczenia.
- `AnfisNet(nn.Module)`: Główna klasa modelu. Metoda `forward` wykonuje pełny przebieg: Fuzzify → Rules → Norm → TSK → Aggregation (przy użyciu `torch.bmm`).

3.4 Moduł: backend/neuro_engine.py

Kontroler procesu uczenia (Trainer).

- `NeuroEngine(class)`
- `train_model(self, df_train, df_future, features, target_col, epochs, lr) -> Generator`: Funkcja generatorowa.
 1. Konwertuje DataFrame na `torch.Tensor`.
 2. Inicjalizuje model `make_anfis` i optymalizator Adam.
 3. Uruchamia pętlę epok z Backpropagation (`loss.backward()`).
 4. Co interwał epok wykonuje `yield json` ze statusem postępu.
 5. Po zakończeniu wykonuje inferencję na zbiorze `df_future`.
 6. Oblicza `future_implied_prices` na podstawie predykcji zwrotu.
- `predict(self, inputs_dict) -> float`: Szybka inferencja dla pojedynczego punktu danych.

3.5 Moduł: backend/backtest_engine.py

Silnik symulacji historycznej.

- `BacktestEngine(class)`
- `run(self, df, buy_threshold, sell_threshold, stop_loss_pct, take_profit_pct, ...) -> dict`: Maszyna stanów iterująca po DataFrame.
 - *States*: OUT_OF_MARKET, IN_POSITION.
 - *Signals*: Otwarcie pozycji gdy `oscillator >= buy_thr`. Zamknięcie gdy `oscillator <= sell_thr` LUB `Price <= SL` LUB `Price >= TP`.
 - *Metrics*: Oblicza Total Return, CAGR, Sharpe Ratio, Max Drawdown, Trades Count.

3.6 Moduł: backend/optimizer_engine.py

(Moduł opcjonalny/legacy) Optymalizacja genetyczna/brute-force parametrów dla statycznego modelu analizy.

- `OptimizerEngine(class)`
- `optimize_weights(self, df, progress_callback) -> dict`: Iteruje po kombinacjach wag dla wskaźników technicznych, szukając zestawu maksymalizującego korelację z przyszłym zwrotem.

4 Specyfikacja API REST (Flask)

Kontroler backend/app.py udostępnia następujące endpointy.

4.1 Grupa: Konfiguracja i Dane

- GET /api/tickers
 - **Opis:** Zwraca listę obsługiwanych instrumentów finansowych.
 - **Response:** JSON List [{symbol, name}].
- POST /api/auto_strategy
 - **Opis:** Automatycznie dobiera zestaw 5 najlepszych wskaźników dla danego tickera.
 - **Body:** {ticker: str}
 - **Response:** JSON Dict (konfiguracja cech).
- POST /api/correlations
 - **Opis:** Oblicza pełną macierz korelacji (Pearson, Spearman, Kendall) dla wszystkich wygenerowanych cech względem targetu.
 - **Response:** JSON Dict (Top 50 cech z wartościami korelacji).

4.2 Grupa: Analiza i Trening

- POST /api/train_neuro
 - **Opis:** Inicjuje proces treningu ANFIS. Endpoint strumieniowy.
 - **Body:** {ticker, features: [], epochs: int}
 - **Response Type:** application/x-json-stream
 - **Format Danych:** Seria obiektów JSON (NDJSON):
 1. {"status": "progress", "percent": int, "loss": float}
 2. ...
 3. {"status": "done", "data": {...results...}}
- POST /api/backtest
 - **Opis:** Uruchamia symulację historyczną na podstawie statycznego modelu wagowego.
 - **Body:** {ticker, config, buyThreshold, sellThreshold, stopLoss, takeProfit}
 - **Response:** JSON Dict (statystyki portfela + krzywa kapitału).
- POST /api/analyze
 - **Opis:** Zwraca szereg czasowy z obliczonym oscylatorem sentymentu (dla wizualizacji bez backtestingu).

4.3 Grupa: Optymalizacja

- POST /api/optimize
 - **Opis:** Uruchamia długotrwały proces poszukiwania optymalnych wag dla wskaźników (Brute Force / Heurystyka).
 - **Response Type:** application/x-json-stream.

5 Funkcje Pomocnicze (Utilities)

5.1 W pliku backend/app.py

- `normalize_rank(series, window=252)`: Normalizuje wskaźnik do przedziału 0-100 na podstawie jego rangi w oknie historycznym (Percentile Rank). Używane do sprowadzenia różnych wskaźników (np. RSI i Wolumen) do wspólnej skali dla systemu Fuzzy Logic.
- `prepare_data_with_features(ticker)`: Wrapper łączący `data_engine` i `feature_factory`, wykonujący dodatkowo czyszczenie NaN i normalizację rangową przed przekazaniem danych do treningu.